Homework1 报告

姓名：朱希涵 | **学号**：SA25219006
日期：March, 2026 | **GPU**：NVIDIA Tesla V100-SXM2-16GB

本人github仓库同步更新：https://github.com/Multipleriver/NN_Homework

1. 摘要

本报告围绕混凝土抗压强度回归任务展开，先按题面要求完成 **初版方案（train.py）**，再基于缺陷进行 **针对性优化复盘（train_modified.py / train_modified_v2.py）**。

初版方案严格使用顺序 80/20 切分、训练集拟合预处理器，并输出 pth/onnx、指标与图表；后续在复盘中按控制变量顺序测试了 Shuffle、MSELoss、增大容量+长训、引入水胶比特征因子四个方向。

初版在测试集表现一般，优化后显著提升；最终 S4 方案（Shuffle+MSELoss+大模型正则+水胶比）获得当前最佳结果（MSE 21.56，R^2 0.9086）。

项目目录树（更新）：

- `NN_Homework/Homework1/project`：作业代码主目录
  - `NN_Homework/Homework1/project/train.py`：初版主线脚本（Step1~11）
  - `NN_Homework/Homework1/project/train_modified.py`：第一轮改进脚本（验证集/早停/Huber）
  - `NN_Homework/Homework1/project/train_modified_v2.py`：本轮控制变量实验脚本（S1~S4）
  - `NN_Homework/Homework1/project/outputs`：初版结果
  - `NN_Homework/Homework1/project/outputs_modified`：第一轮改进结果
  - `NN_Homework/Homework1/project/outputs_modified_v2_s1_shuffle`：S1 结果
  - `NN_Homework/Homework1/project/outputs_modified_v2_s2_shuffle_mse`：S2 结果
  - `NN_Homework/Homework1/project/outputs_modified_v2_s3_capacity_reg`：S3 结果
  - `NN_Homework/Homework1/project/outputs_modified_v2_s4_wb`：S4 结果
  - `NN_Homework/Homework1/project/report`：本 Notebook 报告
- `NN_Homework/Homework1/resource`：题面与原始数据资源

2. 任务定义与约束

- 任务类型：监督学习回归，输入 8 个基础特征，输出 `csMPa`。
- 初版主线约束：
  - 前 80% 训练、后 20% 测试；初版不打乱。

- 输出 MSE、训练损失曲线、真实值-预测值散点图。

3. 方法与实现流程（train.py）

3.1 实现流程概述

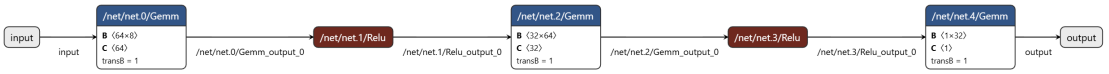
- **Step 1-2**：完成随机种子、路径可访问性与数据合法性检查等前置的操作。
- **Step 3-5**：按题面切分与预处理，随后完成张量化与 DataLoader 构建。
- **Step 6-8**：定义 MLP，配置组件，执行训练。
- **Step 9-11**：测试评估、图像生成、结果汇总。

3.2 开头初始化代码块（imports / 常量 / 模型类 / seed）

In [78]:
```
import argparse
import importlib
import json
import random
from pathlib import Path
from typing import Dict, List, Tuple

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from torch.utils.data import DataLoader, TensorDataset


FEATURE_COLS = [
    "cement",
    "slag",
    "flyash",
    "water",
    "superplasticizer",
    "coarseaggregate",
    "fineaggregate",
    "age",
]
TARGET_COL = "csMPa"


class MLPRegressor(nn.Module):
    def __init__(self, input_dim: int, hidden_dims: Tuple[int, int]) -> None:
        super().__init__()
        h1, h2 = hidden_dims
        self.net = nn.Sequential(
            nn.Linear(input_dim, h1),
            nn.ReLU(),
            nn.Linear(h1, h2),
            nn.ReLU(),
            nn.Linear(h2, 1),
        )
```

```
    def forward(self, x: torch.Tensor) -> torch.Tensor:
        return self.net(x)


def set_seed(seed: int) -> None:
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
```

其中网络（初版）的模型我已在后续代码中导出 .onnx 文件，并手动送进 Netron 渲染，附上：



3.3 Step 1-2：初始化与数据检查

这部分是数据初始化的内容。

关于随机种子，我了解了一下，我的理解就是“伪随机数生成器的起点”，同时固定 random、NumPy、PyTorch CPU、PyTorch CUDA 的随机性（虽然我也不清楚怎么固定的），用来保证结果可复现，更公平。

In [80]:
```
def step1_initialize(project_dir: Path, data_path: Path, seed: int) -> None:
    set_seed(seed)

    print(f"[Step 1] Seed set to: {seed}")
    print(f"[Step 1] Project directory: {project_dir}")
    print(f"[Step 1] Data path: {data_path}")

    if data_path.exists() and data_path.is_file():
        print("[Step 1] Data path check: PASS")
    else:
        raise FileNotFoundError(f"[Step 1] Data path check: FAIL -> {data_path}"


def step2_data_check(data_path: Path) -> pd.DataFrame:
    df = pd.read_csv(data_path)

    print("[Step 2] Data shape:", df.shape)
    print("[Step 2] Columns:", list(df.columns))
    print("[Step 2] Dtypes:")
    print(df.dtypes)

    if df.shape[0] != 1030:
        raise ValueError(f"[Step 2] Sample count mismatch: expected 1030, got {d
    print("[Step 2] Sample count check: PASS (1030)")

    if list(df.columns) != FEATURE_COLS + [TARGET_COL]:
        raise ValueError("[Step 2] Column name/order check: FAIL")
    print("[Step 2] Column check: PASS")

    missing_counts = df.isna().sum()
```

```
    print("[Step 2] Missing value counts:")
    print(missing_counts)

    numeric_cols = df.columns.tolist()
    negative_counts = (df[numeric_cols] < 0).sum()
    print("[Step 2] Negative value counts:")
    print(negative_counts)

    q1 = df[numeric_cols].quantile(0.25)
    q3 = df[numeric_cols].quantile(0.75)
    iqr = q3 - q1
    outlier_mask = (df[numeric_cols] < (q1 - 1.5 * iqr)) | (df[numeric_cols] > (
    outlier_counts = outlier_mask.sum()
    print("[Step 2] IQR outlier counts:")
    print(outlier_counts)

    return df
```

3.4 Step 3：顺序 80/20 切分

按题面进行前 80% 训练、后 20% 测试，并检查样本总数与特征顺序一致性。

In [82]:
```
def step3_split_data(df: pd.DataFrame) -> Dict[str, pd.DataFrame]:
    n_samples = len(df)
    split_idx = int(np.floor(0.8 * n_samples))

    train_df = df.iloc[:split_idx].copy()
    test_df = df.iloc[split_idx:].copy()

    print(f"[Step 3] Total samples: {n_samples}")
    print(f"[Step 3] Split index: {split_idx}")
    print(f"[Step 3] Train samples: {len(train_df)}")
    print(f"[Step 3] Test samples: {len(test_df)}")

    if len(train_df) + len(test_df) != n_samples:
        raise ValueError("[Step 3] Split count check: FAIL")
    if list(train_df[FEATURE_COLS].columns) != list(test_df[FEATURE_COLS].column
        raise ValueError("[Step 3] Feature order consistency check: FAIL")

    print("[Step 3] No shuffle check: PASS (sequential split by index)")
    print("[Step 3] Feature order consistency check: PASS")

    return {
        "train_df": train_df,
        "test_df": test_df,
        "split_idx": split_idx,
    }
```

3.5 Step 4-5：预处理拟合与张量化

Step 4 强调只在训练集上 `fit` 预处理器，Step 5 将数据组织成张量与 DataLoader，作为训练输入。

In [84]:
```
def step4_preprocess(
    train_df: pd.DataFrame, test_df: pd.DataFrame
) -> Dict[str, np.ndarray | SimpleImputer | StandardScaler]:
    x_train_raw = train_df[FEATURE_COLS].values
```

```
    x_test_raw = test_df[FEATURE_COLS].values

    imputer = SimpleImputer(strategy="mean")
    scaler = StandardScaler()

    x_train_imputed = imputer.fit_transform(x_train_raw)
    x_test_imputed = imputer.transform(x_test_raw)

    x_train_scaled = scaler.fit_transform(x_train_imputed)
    x_test_scaled = scaler.transform(x_test_imputed)

    y_train = train_df[TARGET_COL].values.astype(np.float32)
    y_test = test_df[TARGET_COL].values.astype(np.float32)

    if not np.isfinite(x_train_scaled).all() or not np.isfinite(x_test_scaled).a
        raise ValueError("[Step 4] NaN/Inf check: FAIL")

    print("[Step 4] Imputer fit scope: TRAIN only")
    print("[Step 4] Scaler fit scope: TRAIN only")
    print("[Step 4] NaN/Inf check: PASS")
    print(f"[Step 4] X_train shape: {x_train_scaled.shape}, X_test shape: {x_tes

    return {
        "x_train": x_train_scaled.astype(np.float32),
        "x_test": x_test_scaled.astype(np.float32),
        "y_train": y_train,
        "y_test": y_test,
        "imputer": imputer,
        "scaler": scaler,
    }
```

In [86]:

```
def step5_build_tensors_and_loader(
    x_train: np.ndarray,
    y_train: np.ndarray,
    x_test: np.ndarray,
    y_test: np.ndarray,
    batch_size: int,
) -> Dict[str, torch.Tensor | DataLoader]:
    x_train_tensor = torch.tensor(x_train, dtype=torch.float32)
    y_train_tensor = torch.tensor(y_train, dtype=torch.float32).view(-1, 1)
    x_test_tensor = torch.tensor(x_test, dtype=torch.float32)
    y_test_tensor = torch.tensor(y_test, dtype=torch.float32).view(-1, 1)

    train_dataset = TensorDataset(x_train_tensor, y_train_tensor)
    train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True

    sample_x, sample_y = next(iter(train_loader))
    print(f"[Step 5] X_train dtype/shape: {x_train_tensor.dtype}, {tuple(x_train
    print(f"[Step 5] y_train dtype/shape: {y_train_tensor.dtype}, {tuple(y_train
    print(f"[Step 5] X_test dtype/shape: {x_test_tensor.dtype}, {tuple(x_test_te
    print(f"[Step 5] y_test dtype/shape: {y_test_tensor.dtype}, {tuple(y_test_te
    print(f"[Step 5] DataLoader batch sample shape: X={tuple(sample_x.shape)}, y

    return {
        "x_train_tensor": x_train_tensor,
        "y_train_tensor": y_train_tensor,
        "x_test_tensor": x_test_tensor,
        "y_test_tensor": y_test_tensor,
```

```
        "train_loader": train_loader,
    }
```

3.6 Step 6-7：模型构建与训练组件

Step 6 用于模型结构与前向输出检查；Step 7 定义损失与优化器，并配合 `train_one_epoch` 支持单轮或多轮训练。

In [88]:
```
def step6_build_model(input_dim: int, hidden_dims: Tuple[int, int], device: torc
    model = MLPRegressor(input_dim=input_dim, hidden_dims=hidden_dims).to(device
    dummy_x = torch.randn(4, input_dim, device=device)
    dummy_out = model(dummy_x)
    print(f"[Step 6] Model forward output shape check: {tuple(dummy_out.shape)}"
    return model


def step7_setup_training(model: nn.Module, learning_rate: float) -> Tuple[nn.Mod
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    print(f"[Step 7] Loss function: {criterion.__class__.__name__}")
    print(f"[Step 7] Optimizer: Adam, lr={learning_rate}")
    return criterion, optimizer


def train_one_epoch(
    model: nn.Module,
    loader: DataLoader,
    criterion: nn.Module,
    optimizer: torch.optim.Optimizer,
    device: torch.device,
) -> float:
    model.train()
    total_loss = 0.0
    n_samples = 0

    for batch_x, batch_y in loader:
        batch_x = batch_x.to(device)
        batch_y = batch_y.to(device)

        optimizer.zero_grad()
        preds = model(batch_x)
        loss = criterion(preds, batch_y)
        loss.backward()
        optimizer.step()

        total_loss += loss.item() * batch_x.size(0)
        n_samples += batch_x.size(0)

    return total_loss / max(n_samples, 1)
```

3.7 Step 8：训练循环、模型保存与 ONNX 自检

这一步是主训练环节，包含 epoch 循环、权重保存、ONNX 导出与 ONNXRuntime 一致性检查。

In [90]:

```
def onnx_inference_self_check(
    model: nn.Module,
    onnx_path: Path,
    sample_input: torch.Tensor,
    atol: float = 1e-4,
    rtol: float = 1e-3,
) -> Dict[str, float]:
    try:
        ort = importlib.import_module("onnxruntime")
    except ImportError as exc:
        raise ImportError(
            "ONNX self-check requires onnxruntime. Install it with: pip install
        ) from exc

    model.eval()
    with torch.no_grad():
        torch_out = model(sample_input).detach().cpu().numpy()

    ort_session = ort.InferenceSession(str(onnx_path), providers=["CPUExecutionP
    ort_inputs = {ort_session.get_inputs()[0].name: sample_input.detach().cpu().
    ort_out = ort_session.run(None, ort_inputs)[0]

    max_abs_diff = float(np.max(np.abs(torch_out - ort_out)))
    passed = bool(np.allclose(torch_out, ort_out, atol=atol, rtol=rtol))

    if not passed:
        raise RuntimeError(
            f"ONNX self-check failed: max_abs_diff={max_abs_diff:.6e}, atol={ato
        )

    return {
        "max_abs_diff": max_abs_diff,
        "atol": float(atol),
        "rtol": float(rtol),
    }


def step8_train_and_save(
    model: nn.Module,
    loader: DataLoader,
    criterion: nn.Module,
    optimizer: torch.optim.Optimizer,
    device: torch.device,
    epochs: int,
    output_dir: Path,
    loss_history: List[float],
) -> Dict[str, Path | List[float]]:
    for epoch in range(len(loss_history) + 1, epochs + 1):
        epoch_loss = train_one_epoch(model, loader, criterion, optimizer, device
        loss_history.append(epoch_loss)
        if epoch % 20 == 0 or epoch == 1 or epoch == epochs:
            print(f"[Step 8] Epoch {epoch:03d}/{epochs}, Train Loss: {epoch_loss

    model_path = output_dir / "final_model.pth"
    torch.save(model.state_dict(), model_path)

    onnx_path = output_dir / "final_model.onnx"
    model.eval()
    dummy_input = torch.randn(1, loader.dataset.tensors[0].shape[1], device=devi
```

```
    torch.onnx.export(
        model,
        dummy_input,
        onnx_path,
        export_params=True,
        opset_version=17,
        do_constant_folding=True,
        input_names=["input"],
        output_names=["output"],
        dynamic_axes={"input": {0: "batch_size"}, "output": {0: "batch_size"}},
    )

    check_result = onnx_inference_self_check(
        model=model,
        onnx_path=onnx_path,
        sample_input=dummy_input,
    )

    loss_path = output_dir / "train_loss_history.json"
    with open(loss_path, "w", encoding="utf-8") as f:
        json.dump({"train_loss": loss_history}, f, indent=2)

    # Quick load test for completion criteria.
    reloaded_model = MLPRegressor(input_dim=loader.dataset.tensors[0].shape[1],
    reloaded_model.load_state_dict(torch.load(model_path, map_location="cpu", we
    print(f"[Step 8] Model saved: {model_path}")
    print(f"[Step 8] ONNX saved: {onnx_path}")
    print(
        "[Step 8] ONNX self-check: PASS "
        f"(max_abs_diff={check_result['max_abs_diff']:.6e}, "
        f"atol={check_result['atol']:.1e}, rtol={check_result['rtol']:.1e})"
    )
    print(f"[Step 8] Loss history saved: {loss_path}")
    print("[Step 8] Model reload check: PASS")

    return {
        "model_path": model_path,
        "onnx_path": onnx_path,
        "loss_path": loss_path,
        "loss_history": loss_history,
    }
```

3.8 Step 9-10：测试评估与可视化

Step 9 统一计算 MSE/RMSE/MAE/R^2 并保存 JSON；Step 10 输出损失曲线与真实值-预测值散点图。

In [92]:
```
def step9_evaluate(
    model: nn.Module,
    x_test: torch.Tensor,
    y_test: torch.Tensor,
    device: torch.device,
    output_dir: Path,
) -> Dict[str, np.ndarray | float | Path]:
    model.eval()
    with torch.no_grad():
        preds = model(x_test.to(device)).cpu().numpy().reshape(-1)
```

```
    y_true = y_test.cpu().numpy().reshape(-1)

    mse = mean_squared_error(y_true, preds)
    rmse = float(np.sqrt(mse))
    mae = mean_absolute_error(y_true, preds)
    r2 = r2_score(y_true, preds)

    metrics = {
        "mse": float(mse),
        "rmse": float(rmse),
        "mae": float(mae),
        "r2": float(r2),
    }
    metrics_path = output_dir / "test_metrics.json"
    with open(metrics_path, "w", encoding="utf-8") as f:
        json.dump(metrics, f, indent=2)

    print(f"[Step 9] MSE: {metrics['mse']:.6f}")
    print(f"[Step 9] RMSE: {metrics['rmse']:.6f}")
    print(f"[Step 9] MAE: {metrics['mae']:.6f}")
    print(f"[Step 9] R2: {metrics['r2']:.6f}")
    print(f"[Step 9] Metrics saved: {metrics_path}")

    return {
        "y_true": y_true,
        "y_pred": preds,
        "metrics": metrics,
        "metrics_path": metrics_path,
    }


def step10_visualize(loss_history: List[float], y_true: np.ndarray, y_pred: np.n
    loss_fig = output_dir / "train_loss_curve.png"
    scatter_fig = output_dir / "test_true_vs_pred.png"

    plt.figure(figsize=(8, 5))
    plt.plot(np.arange(1, len(loss_history) + 1), loss_history, label="Train Los
    plt.xlabel("Epoch")
    plt.ylabel("MSE Loss")
    plt.title("Training Loss Curve")
    plt.legend()
    plt.tight_layout()
    plt.savefig(loss_fig, dpi=150)
    plt.close()

    plt.figure(figsize=(6, 6))
    plt.scatter(y_true, y_pred, alpha=0.7, label="Predictions")
    min_val = min(float(y_true.min()), float(y_pred.min()))
    max_val = max(float(y_true.max()), float(y_pred.max()))
    plt.plot([min_val, max_val], [min_val, max_val], "r--", label="Ideal y=x")
    plt.xlabel("True Strength (MPa)")
    plt.ylabel("Predicted Strength (MPa)")
    plt.title("True vs Predicted on Test Set")
    plt.legend()
    plt.tight_layout()
    plt.savefig(scatter_fig, dpi=150)
    plt.close()

    print(f"[Step 10] Saved loss curve: {loss_fig}")
    print(f"[Step 10] Saved scatter plot: {scatter_fig}")
```

```
    return {
        "loss_curve_path": loss_fig,
        "scatter_plot_path": scatter_fig,
    }
```

3.9 Step 11 与主程序收尾（run_pipeline / parse_args / main）

由于 Jupyter notebook 真的很用不惯，.py能运行但是放到 Jupyter 就不行,稍微调整了一下。

下一次作业我不想用 Jupyter 写了，还是老老实实用 Markdown 吧，有一些需要标注的内容同步注释到代码里面，可能这样会比较直观。

In [94]:
```
def step11_finalize_summary(
    output_dir: Path,
    seed: int,
    split_idx: int,
    train_count: int,
    test_count: int,
    hidden_dims: Tuple[int, int],
    learning_rate: float,
    batch_size: int,
    epochs: int,
    metrics: Dict[str, float],
    model_path: Path,
    onnx_path: Path,
    loss_curve_path: Path,
    scatter_plot_path: Path,
) -> Path:
    summary_path = output_dir / "final_summary.txt"
    lines = [
        "Homework1 Baseline Summary",
        "==========================",
        f"Seed: {seed}",
        f"Split policy: first 80% train / last 20% test (no shuffle)",
        f"Split index: {split_idx}",
        f"Train count: {train_count}",
        f"Test count: {test_count}",
        f"Features: {FEATURE_COLS}",
        f"Target: {TARGET_COL}",
        f"Model: MLP hidden dims={hidden_dims}",
        f"Learning rate: {learning_rate}",
        f"Batch size: {batch_size}",
        f"Epochs: {epochs}",
        f"Test MSE: {metrics['mse']:.6f}",
        f"Test RMSE: {metrics['rmse']:.6f}",
        f"Test MAE: {metrics['mae']:.6f}",
        f"Test R2: {metrics['r2']:.6f}",
        f"Model path: {model_path}",
        f"ONNX path: {onnx_path}",
        f"Loss curve path: {loss_curve_path}",
        f"Scatter plot path: {scatter_plot_path}",
        "Reproducibility notes:",
        "- Fixed random seed for random/NumPy/PyTorch.",
        "- Preprocessing fit only on training set, then transform test set.",
        "- Execution command: .conda/nn_hw/python.exe Homework1/project/train.py
```

```
    ]
    summary_text = "\n".join(lines)
    with open(summary_path, "w", encoding="utf-8") as f:
        f.write(summary_text)

    print(f"[Step 11] Final summary saved: {summary_path}")
    return summary_path


def run_pipeline(args: argparse.Namespace) -> None:
    project_dir = Path(__file__).resolve().parent
    data_path = Path(args.data_path)
    output_dir = project_dir / "outputs"
    output_dir.mkdir(parents=True, exist_ok=True)

    if not (1 <= args.step <= 11):
        raise ValueError("--step must be in [1, 11]")

    step1_initialize(project_dir=project_dir, data_path=data_path, seed=args.see
    if args.step == 1:
        return

    df = step2_data_check(data_path=data_path)
    if args.step == 2:
        return

    split_artifacts = step3_split_data(df)
    train_df = split_artifacts["train_df"]
    test_df = split_artifacts["test_df"]
    if args.step == 3:
        return

    preprocess_artifacts = step4_preprocess(train_df=train_df, test_df=test_df)
    if args.step == 4:
        return

    tensor_artifacts = step5_build_tensors_and_loader(
        x_train=preprocess_artifacts["x_train"],
        y_train=preprocess_artifacts["y_train"],
        x_test=preprocess_artifacts["x_test"],
        y_test=preprocess_artifacts["y_test"],
        batch_size=args.batch_size,
    )
    if args.step == 5:
        return

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"[Info] Device selected: {device}")

    model = step6_build_model(
        input_dim=tensor_artifacts["x_train_tensor"].shape[1],
        hidden_dims=(args.hidden_dim1, args.hidden_dim2),
        device=device,
    )
    if args.step == 6:
        return

    criterion, optimizer = step7_setup_training(model=model, learning_rate=args.
    loss_history: List[float] = []
    one_epoch_loss = train_one_epoch(
```

```
        model=model,
        loader=tensor_artifacts["train_loader"],
        criterion=criterion,
        optimizer=optimizer,
        device=device,
    )
    loss_history.append(one_epoch_loss)
    print(f"[Step 7] Single-epoch loss: {one_epoch_loss:.6f}")
    if args.step == 7:
        return

    train_artifacts = step8_train_and_save(
        model=model,
        loader=tensor_artifacts["train_loader"],
        criterion=criterion,
        optimizer=optimizer,
        device=device,
        epochs=args.epochs,
        output_dir=output_dir,
        loss_history=loss_history,
    )
    if args.step == 8:
        return

    eval_artifacts = step9_evaluate(
        model=model,
        x_test=tensor_artifacts["x_test_tensor"],
        y_test=tensor_artifacts["y_test_tensor"],
        device=device,
        output_dir=output_dir,
    )
    if args.step == 9:
        return

    figure_artifacts = step10_visualize(
        loss_history=train_artifacts["loss_history"],
        y_true=eval_artifacts["y_true"],
        y_pred=eval_artifacts["y_pred"],
        output_dir=output_dir,
    )
    if args.step == 10:
        return

    step11_finalize_summary(
        output_dir=output_dir,
        seed=args.seed,
        split_idx=split_artifacts["split_idx"],
        train_count=len(train_df),
        test_count=len(test_df),
        hidden_dims=(args.hidden_dim1, args.hidden_dim2),
        learning_rate=args.learning_rate,
        batch_size=args.batch_size,
        epochs=args.epochs,
        metrics=eval_artifacts["metrics"],
        model_path=train_artifacts["model_path"],
        onnx_path=train_artifacts["onnx_path"],
        loss_curve_path=figure_artifacts["loss_curve_path"],
        scatter_plot_path=figure_artifacts["scatter_plot_path"],
    )
```

```python
def parse_args() -> argparse.Namespace:
    parser = argparse.ArgumentParser(description="Homework1 regression pipeline"
    parser.add_argument("--step", type=int, default=1, help="Current step to exe
    parser.add_argument("--seed", type=int, default=42, help="Random seed")
    parser.add_argument("--learning-rate", type=float, default=1e-3, help="Adam
    parser.add_argument("--batch-size", type=int, default=32, help="Training bat
    parser.add_argument("--epochs", type=int, default=200, help="Training epochs
    parser.add_argument("--hidden-dim1", type=int, default=64, help="First hidde
    parser.add_argument("--hidden-dim2", type=int, default=32, help="Second hidd
    parser.add_argument(
        "--data-path",
        type=str,
        default=str(
            (Path(__file__).resolve().parents[1] / "resource" / "Concrete_Data_Y
            if "__file__" in globals()
            else (Path.cwd() / "Homework1" / "resource" / "Concrete_Data_Yeh.csv
        ),
        help="Path to Concrete_Data_Yeh.csv",
    )
    if "__file__" in globals():
        return parser.parse_args()
    return parser.parse_args(args=[])


def main() -> None:
    args = parse_args()
    run_pipeline(args)


if __name__ == "__main__":
    if "__file__" in globals():
        main()
    else:
        print("[Info] Notebook mode detected: skip main() auto-execution.")
```

```
[Info] Notebook mode detected: skip main() auto-execution.
```

4. 初步结果分析

4.1 初版方案的数据处理方式

- 按原始顺序切分：前 80% 训练、后 20% 测试。
- 训练集拟合 `SimpleImputer(mean)` 与 `StandardScaler`，测试集仅 `transform`。
- 标签 `y` 直接用原始尺度训练，不做目标标准化。

4.2 初版网络结构

- 模型：`MLPRegressor`，结构为 `8 -> 64 -> 32 -> 1`。
- 激活函数：ReLU。
- 损失函数：MSELoss。
- 优化器：Adam，学习率 `1e-3`，epoch=200。

4.3 缺陷

- **测试集偏差较大**：点云偏离对角线较明显，部分区间出现系统偏差。
- **泛化不稳定**：训练损失下降较好，但测试指标提升有限，应该是泛化不足。

5. 针对性方案优化（train_modified.py / train_modified_v2.py）

我没有一上来就堆很多技巧，数据集的数量也不是很多，所以找到主要的矛盾点比较重要。

1. 先做 **Shuffle**，关注分布偏移；
2. 把损失切成 **MSELoss** 以优化均方指标；
3. 在更稳的数据划分和损失下，做 **容量提升 + 正则化 + 长训练**；
4. 因为最近对自动化金融决策和交易比较感兴趣，想到也可以人工构造一些因子来找规律，调查之后加入一个土木领域的特征**水胶比**。

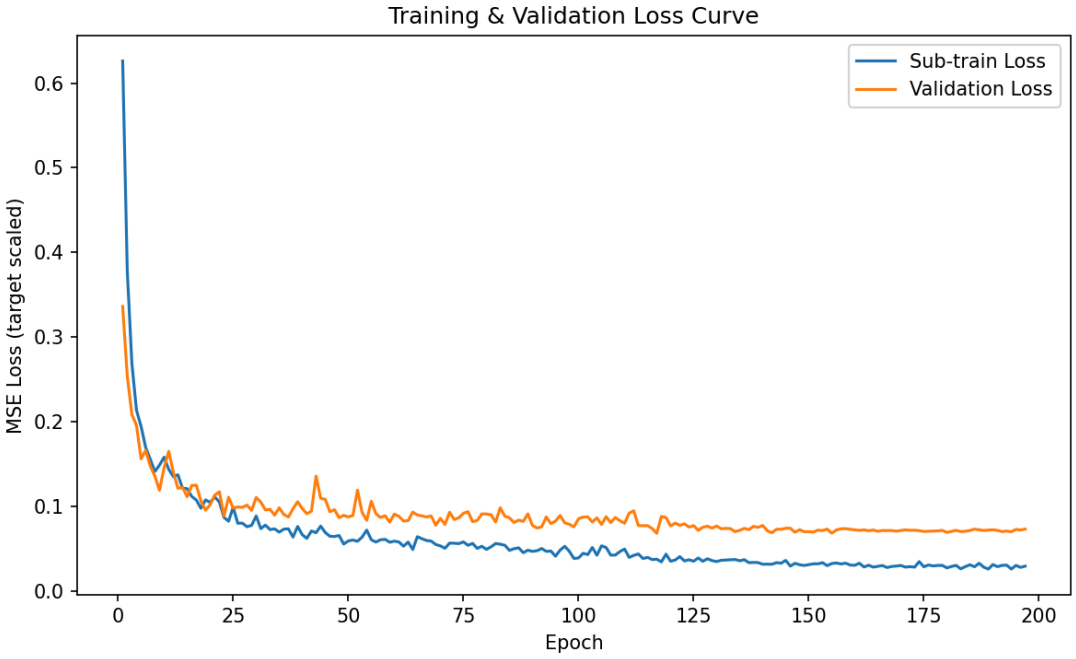
5.1 控制变量复盘与结论

- **S1（仅 Shuffle）** 就带来跨越式提升，说明此前主要痛点是训练/测试分布偏移；
- **S2（S1 + MSELoss）** 在均方指标上继续小幅改善；
- **S3（S2 + 大容量+正则+长训练）** 进一步提高精度，泛化更稳；
- **S4（S3 + 水胶比特征）** 取得当前最优。

对比：

实验	关键改动	MSE	RMSE	MAE	R^2
Modified	不打乱 + Huber + 小模型	105.5104	10.2718	8.6236	0.3087
S1_Shuffle	仅 Shuffle	23.7615	4.8746	3.6232	0.8993
S2_Shuffle_MSE	S1 + MSELoss	23.4871	4.8464	3.6057	0.9004
S3_S2_plus_Capacity_Reg	S2 + 大容量+正则+长训	21.9920	4.6896	3.4289	0.9068
S4_S3_plus_WaterBinder	S3 + 水胶比	21.5582	4.6431	3.3405	0.9086

S4 最终结果



先把主要矛盾找出来，再一层层收敛，看着散点图从松散到靠近对角线还是比较有成就感。

6. 结果总结

初版 `train.py` 严格按题面约束实现了完整主线，但测试集仍有明显偏差；通过分步复盘（Shuffle -> MSELoss -> 增容正则长训 -> 水胶比特征，主要是Shuffle），模型性能持续改善。

当前最优是 S4： `MSE=21.5582, RMSE=4.6431, MAE=3.3405, R^2=0.9086` 。